Custom CacheManager & Granular TTL

The Problem: Global TTLs & Redis Out-Of-Memory (OOM)

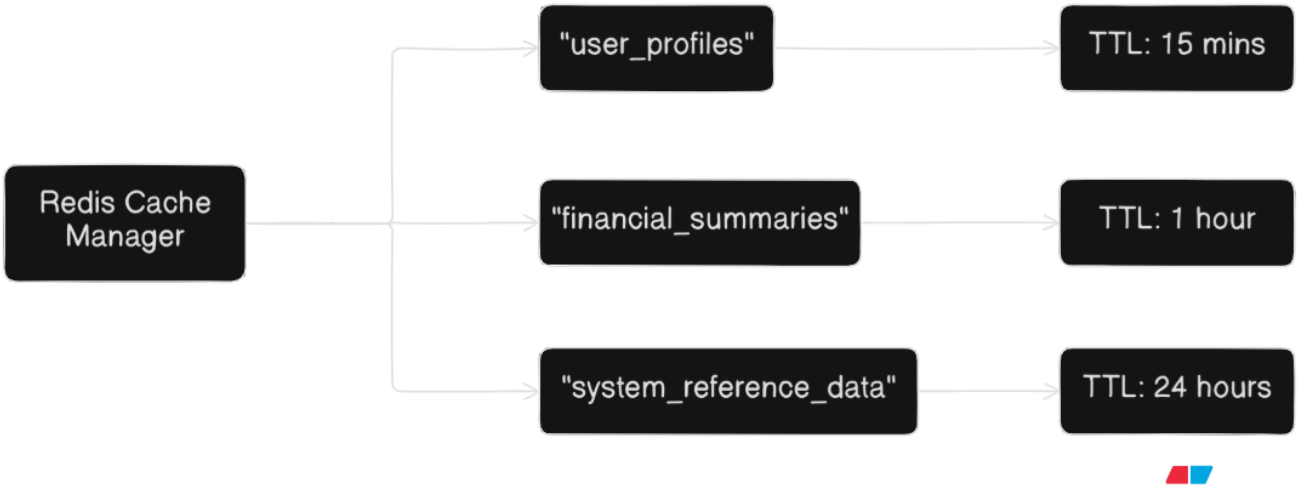
In a default Spring Boot setup, setting a single global Time-To-Live (TTL) for all caches causes significant operational issues:

1. **Memory Bloat & Eviction Thrashing:** Static, rarely updated reference data (e.g., country codes) shares the same expiration window as volatile, high-frequency data (e.g., active user sessions or stock prices).
2. **OOM Crashes:** Without TTL-driven memory management, non-expiring key growth eventually triggers Redis's `maxmemory` limit. Redis will either crash or start evicting critical keys under `noeviction` or volatile eviction policies.

> A production-grade Spring Boot application requires **granular, per-cache name TTL configuration** mapped cleanly within a custom `RedisCacheManager`.

Key Strategy: Granular TTL Mapping Architecture

Instead of applying a single default expiry, build a dynamic `RedisCacheManager` that assigns specific TTL durations per cache name:



Production Implementation: Custom `RedisCacheManager` Builder

Use Spring's `RedisCacheManagerBuilderCustomizer` or explicit `RedisCacheManager` configuration to define per-cache expiration policies.

```
@Configuration
Hpublic class DynamicTtlCacheConfig {

    // Define cache name constants to avoid magic strings across services
    public static final String CACHE_USER_PROFILES = "user_profiles";
```

```
    public static final String CACHE_FINANCIAL_SUMMARIES =
"financial_summaries";
    public static final String CACHE_REFERENCE_DATA =
"system_reference_data";

    /**
     * Customizer bean that injects per-cache TTL maps into the auto-
configured RedisCacheManager.
     */
    @Bean
    public RedisCacheManagerBuilderCustomizer
redisCacheManagerBuilderCustomizer(
            GenericJackson2JsonRedisSerializer serializer) {

        // Base configuration applied across all caches
        RedisCacheConfiguration defaultConfig =
RedisCacheConfiguration.defaultCacheConfig()
                .entryTtl(Duration.ofMinutes(30)) // Fallback TTL for
unnamed/new caches
                .disableCachingNullValues()

.serializeKeysWith(RedisSerializationContext.SerializationPair.fromSeriali
zer(new StringRedisSerializer()))

.serializeValuesWith(RedisSerializationContext.SerializationPair.fromSeria
lizer(serializer));

        return builder -> {
            Map<String, RedisCacheConfiguration> cacheConfigurations = new
HashMap<>();

            // 1. High-volatility data: Short TTL to conserve RAM
            cacheConfigurations.put(
                    CACHE_USER_PROFILES,
                    defaultConfig.entryTtl(Duration.ofMinutes(15))
            );

            // 2. Medium-volatility data: Standard operational TTL
            cacheConfigurations.put(
                    CACHE_FINANCIAL_SUMMARIES,
                    defaultConfig.entryTtl(Duration.ofHours(1))
            );

            // 3. Low-volatility reference data: Long TTL
            cacheConfigurations.put(
                    CACHE_REFERENCE_DATA,
                    defaultConfig.entryTtl(Duration.ofDays(1))
            );

            builder.cacheDefaults(defaultConfig)
```

```
                    .withInitialCacheConfigurations(cacheConfigurations);
        };
    }
}
```

Usage in Business Logic

Inject constants into your Spring `@Cacheable` service annotations to ensure type safety:

```
@Service
@RequiredArgsConstructor
public class AccountService {

    private final AccountRepository accountRepository;

    // Assigned 1-Hour TTL via CACHE_FINANCIAL_SUMMARIES config
    @Cacheable(value = DynamicTtlCacheConfig.CACHE_FINANCIAL_SUMMARIES,
key = "#accountId")
    public FinancialSummaryDTO getFinancialSummary(Long accountId) {
        return
accountRepository.findAggregatedSummaryByAccountId(accountId)
                .map(FinancialSummaryDTO::fromEntity)
                .orElse(null);
    }
}
```

Verifying TTLs via Redis CLI

Inspect the remaining time-to-live for generated keys in the CLI to verify your per-cache configurations are applying correctly:

```
# 1. Fetch active keys
127.0.0.1:6379> KEYS *
1) "financial_summaries::101"
2) "user_profiles::2045"

# 2. Check remaining TTL (Returns value in seconds)
127.0.0.1:6379> TTL "financial_summaries::101"
(integer) 3542  # ~59 minutes remaining (1-Hour TTL Applied)

127.0.0.1:6379> TTL "user_profiles::2045"
(integer) 840   # ~14 minutes remaining (15-Min TTL Applied)
```

Architectural & Production Common Problems

1. Cache Stampede (Thundering Herd) on Expiration

When a high-traffic key (e.g., `CACHE_FINANCIAL_SUMMARIES`) expires after its 1-hour TTL, hundreds of concurrent threads might hit a Cache Miss simultaneously. All threads then execute the expensive DB query at the same time, overwhelming your database.

> **Fix:** Enable **`sync = true`** inside `@Cacheable`. This locks local method execution per key so only **one** thread queries the DB on a miss, while parallel requests wait for the cache to populate:

```
@Cacheable(
    value = DynamicTtlCacheConfig.CACHE_FINANCIAL_SUMMARIES,
    key = "#accountId",
    sync = true
)
public FinancialSummaryDTO getFinancialSummary(Long accountId) { ... }
```

2. Redis Memory Max Eviction Alignment

Even with custom TTLs, ensure your `redis.conf` sets an explicit memory ceiling and eviction policy:

```
maxmemory 2gb
maxmemory-policy volatile-lru  # Removes keys with an expired/active TTL first when RAM fills up
```

3. Misspelled Cache Names

If a developer uses `@Cacheable(value = "financial_summary")` (missing the 's'), Spring won't throw an error. It will fall back to `defaultConfig` (30 mins) instead of the intended custom 1-hour TTL.

> **Fix:** Enforce `public static final String` constants across all project modules.